10 — My Common Mistakes (check this before and after every problem)

⚠️ =+ IS NOT += ⚠️

This is my #1 bug. It has cost me on `sum_odd`, `count_letter`, `grid_total`, `most_frequent`, and `all_longest_strings`.

```
x = 10
x += 5      #  x is 15   <- ADD to x        (plus FIRST)
x =+ 5      #  x is 5    <- means x = +5    (plus SECOND = plain assignment)
```

`+5` just means "positive 5", exactly like `-5` means negative 5. So `x =+ 5` throws away the old value.

Python NEVER warns you. No error, no red underline. The code runs and silently gives a wrong answer — and it often passes the one visible example by luck.

The rule: the plus touches the variable. `x += 5` ✓ · `x =+ 5` ✗ Same for `-=`, `*=`, `//=`.

Every time I type `=`, check whether a `+` belongs before it.

⚠️ WHEN DO I NEED `len()`? ⚠️

`range` always needs a NUMBER. A list is not a number. So: `range(len(...))`

```
for r in range(len(grid)):        # ✓ len needed — range wants a COUNT
for r in range(grid):             # ✗ CRASH — grid is a list, not a number
```

Treat `range(len(` as ONE unit — they go together.

NO `len()` needed anywhere else:

```
grid[r][c]         # r and c are already numbers
grid[0]            # first row
nums[-1]           # last item
nums[1:4]          # slicing
for x in nums      # walking VALUES — no positions at all
result.append(x)   # adding
```

The only other two places `len()` appears:

```
if 0 <= r < len(grid):    # bounds check
return len(nums)          # "how many"
```

Quick test: am I typing `range(`? Then `len(` probably comes right after.

Tip: compute sizes once at the top so you type it less — `rows = len(grid)` / `cols = len(grid[0])` — then `for r in range(rows):`

These are **my own** repeating mistakes, collected from real practice sessions. Read the checklist before starting. Run the checklist again before submitting.

THE PRE-SUBMIT CHECKLIST (30 seconds, do it every time)

- ☐ Does my function `return` something? (Not `print`, not nothing.)
- ☐ Is the `return` **after** the loop? (Unless I'm searching and stopping early.)
- ☐ Did I use the **parameter** given to me? (No hardcoded values, no `input()`.)
- ☐ Is my setup line (`total = 0`, `result = []`) **before** the loop, not inside it?
- ☐ Is the whole body **indented** under `def`?
- ☐ Did I assign the result of `.append()` or `.sort()` by accident? (Both return `None`.)
- ☐ Does it work on a **second, different** example — not just the given one?

MISTAKE 1 — `return` inside a collecting loop

What I do wrong:

```
for x in nums:
    result.append(x)
    return result      # WRONG — exits on the FIRST item
```

Why it's wrong: `return` ends the function immediately. The loop runs once.

The rule:

- **Collecting/summing/counting?** → `return` goes **after** the loop.
- **Searching for one thing and stopping?** → `return` inside the loop is **correct**.

How I catch it: if my answer only reflects the first item, this is why.

MISTAKE 2 — Hardcoding over the parameter

What I do wrong:

```
def total(nums):
```

```
    nums = [1, 2, 3]      # WRONG — throws away what was passed in
```

Or asking for `input()` when the value already arrived as a parameter.

Why it's wrong: the test passes real data in. Overwriting it means my function only ever works for one input — and `input()` makes it hang forever on the exam.

The rule: the value **arrives** in the parameter. Use it. Never redefine it, never ask for it.

MISTAKE 3 — The 3 zones (setup / work / return)

What I do wrong: putting a setup line inside the loop, so it resets every turn.

```
for x in nums:
    result = []           # WRONG — wipes the list on every item
    result.append(x)
```

The rule — every collecting loop has three zones:

```
result = []          ZONE 1 — BEFORE the loop: set up ONCE
for x in nums:       ZONE 2 — INSIDE the loop: repeat per item
    result.append(x)
return result        ZONE 3 — AFTER the loop: finish ONCE
```

How I catch it: indentation tells me the zone. More indented than `for` = inside the loop.

MISTAKE 4 — `len(list)` vs `len(item)`

What I do wrong: using `len(words)` when I meant the length of a word.

Expression	Means	For `["abc", "de"]`
`len(words)`	how many **items**	`2`
`len(words[0])`	how long the **first item** is	`3`
`len(w)` inside `for w in words`	how long **this** item is	`3`, then `2`

How I catch it: say out loud "am I counting items, or measuring one item?"

MISTAKE 5 — Methods that return `None`

What I do wrong:

```
new = nums.append(5)      # WRONG — new is None
nums = nums.sort()        # WRONG — nums becomes None
```

Why: these change the list **in place** and hand back nothing.

Return `None`: `.append()` `.sort()` `.reverse()` `.insert()` `.remove()` `.add()` **Return a new value:** `sorted()` `sum()` `len()` `max()` `.upper()` `.split()` `.join()`

The rule: in-place methods are **called**, never **assigned**.

MISTAKE 6 — Function shape vs method shape

What I do wrong: `a.sorted()`, `a.len()`

The rule — two different shapes:

```
sorted(a)   len(a)   sum(a)   max(a)   min(a)   abs(n)     # FUNCTION wraps the thing
a.upper()   a.split()   a.append(x)   a.strip()           # METHOD attaches with a dot
```

There is no `a.sorted()` and no `a.len()`. If unsure, check 01d-operations-and-builtins.md.

MISTAKE 7 — Indentation / empty function body

What I do wrong: writing the body flush-left, so it's outside the function.

Why it's wrong: the function ends up empty and returns `None`. A `return` outside a function is a `SyntaxError`.

The rule: everything belonging to the function is **indented** under `def`. Everything belonging to the loop is indented one level further.

MISTAKE 8 — `=+` is not `+=`

What I do wrong: `longest =+ len(w)`

Why it's evil: `x =+ 5` means `x = +5` — a plain assignment. **Python does not error**, it just silently does the wrong thing.

The rule:

- Add to it → `+=` (plus **first**)
- Replace it → `=` (plain)
- Best-so-far always uses plain `=`, never `+=`.

MISTAKE 9 — Missing quotes on text

What I do wrong: `a = listen`

Why: without quotes Python thinks `listen` is a variable name → `NameError`. Text always needs quotes: `"listen"`.

MISTAKE 10 — Testing only the given example

What I do wrong: it passes the one example, so I assume it's right.

Real case: `longest += len(w)` passed `["aba","aa","ad","vcd","aba"]` by luck (longest word came first) but broke on `["aa","aba"]`.

The rule — always test a second input:

- smallest: `[]`, one item, `0`
- reversed order of the given example
- all-same values: `[5,5,5]`, `"aaa"`

ERROR MESSAGE DECODER

Read the error **before** asking anyone. It names the line and the problem.

Error message	What it actually means	Where to look
`NameError: name 'x' is not defined`	Used a name that doesn't exist — typo, or missing quotes on text	Check spelling; add `"` around text
`IndentationError` / `unexpected indent`	Lines aren't lined up	Mistake 7
`SyntaxError: invalid syntax`	Python can't parse the line — often a missing `:`, `)`, or `elif` with no condition	The line named, **or the line above it**
`AttributeError: 'int' object has no attribute 'append'`	Called a list method on a non-list (e.g. `result = 0` then `.append()`)	Mistake 5 / 6
`TypeError: 'int' object is not iterable`	Tried to loop over a single number instead of a list	Check what you're looping over
`TypeError: object of type 'int' has no len()`	Called `len()` on a number	Mistake 4
`IndexError: list index out of range`	Asked for a position that doesn't exist — usually `nums[i+1]` on the last item	Loop to `len(nums) - 1`
`ZeroDivisionError`	Divided or `%`'d by zero	Guard the zero case first

Error message	What it actually means	Where to look
`KeyError: 'x'`	Asked a dict for a missing key	Use `.get(key, default)`
Returns `None` unexpectedly	No `return` ran — either missing, or the function fell off the end	Mistakes 1, 3, 7

THE SILENT BUGS (no error, wrong answer) — the dangerous ones

These don't crash, so only testing catches them:

1. `=+` instead of `=` or `+=`
2. `return` inside the loop → only the first item is processed
3. Setup inside the loop → resets each turn
4. `>` where I meant `==` (finding the max vs collecting all matches)
5. `range(1, n)` where I meant `range(1, n + 1)` → misses the last value
6. Comparing the originals instead of the transformed versions

The only defence is Step 6 of 00-how-to-solve-any-problem.md: attack my own solution with a second example before submitting.